

MUSIC CLOSER — ANTIGRAVITY MASTER DEVELOPMENT PROMPT

MUSIC CLOSER — ANTIGRAVITY MASTER DEVELOPMENT PROMPT

You are a senior full-stack engineer, audio-engine developer, music-software architect, and UI/UX designer.

Build a polished, production-quality web application called MUSIC CLOSER.

1. PRODUCT VISION

Music Closer is a browser-based interactive music platform designed to make music accessible, enjoyable, and emotionally engaging.

Core experience:

LISTEN → PLAY → CREATE → ARRANGE → SAVE → SHARE

The application combines:

- Virtual musical instruments
- A lightweight DAW/music-production workspace inspired by FL Studio
- A piano-roll note editor
- A pattern/track-based sequencer
- Chord and music-theory assistance
- A music player
- A Play Along experience
- Local project saving
- Project import/export
- Audio/MIDI/project export

IMPORTANT:

Do NOT clone FL Studio. Keep the experience simpler and more approachable.

Primary goal:

"Let someone open the website and immediately feel close to music."

Prioritize the joy of playing and creating over professional DAW complexity.

2. TARGET EXPERIENCE

A new user should be able to:

1. Open the website.
2. Click a piano key.
3. Immediately hear a piano.
4. Play using their computer keyboard.
5. Record their performance.
6. See notes appear in a piano roll.
7. Move/edit those notes.
8. Press play.
9. Hear the edited composition.
10. Add drums/bass.
11. Arrange parts.
12. Save the project locally.

This core loop is more important than advanced features.

3. TECH STACK

Frontend:

- React
- TypeScript
- Vite
- Tailwind CSS

Audio:

- Tone.js
- Web Audio API where necessary

State:

- Zustand

Persistence:

- IndexedDB preferred for project data
- localStorage for lightweight settings/preferences

Icons:

- Lucide React

Rendering:

- HTML Canvas or SVG for piano roll/timeline where appropriate.

IMPORTANT:

There is NO authentication system.
There are NO user accounts.
There is NO login/signup.
There is NO Supabase requirement.
There is NO cloud synchronization requirement.
The app must be usable immediately without an account.

4. HIGH-LEVEL ARCHITECTURE

Separate:

UI
Music Engine
Data
Audio Engine

CRITICAL:

Musical data must remain independent from audio rendering.

Example:

A minor → A + C + E → selected instrument → sampler/synth → audio

Do NOT represent chords as individual audio files.

5. APPLICATION STRUCTURE

Major areas:

/app/studio
/app/instruments
/app/player
/app/projects
/app/settings

Suggested components:

AppShell
TopBar
InstrumentPanel
PianoKeyboard
PianoRoll
Timeline
Track
Pattern
TransportControls
Mixer
ChordPanel
ScalePanel
Metronome
MusicPlayer
ProjectBrowser
ExportDialog
DrumMachine

6. MAIN UI

Use a modern lightweight DAW layout.

Top bar:

Music Closer
[New] [Open] [Save] [Export]
Project Name

Left sidebar:

INSTRUMENTS
Piano
Guitar
Bass
Drums
Synth

TOOLS

Chords
Scales
Mixer
Player

Center:

Piano Roll / Arrangement

Bottom:

Play / Pause / Record / Stop / Loop
BPM
Metronome

Visual direction:

- Dark studio aesthetic
- Modern
- Minimal
- Elegant
- Soft rounded corners
- Subtle borders
- Clear hierarchy
- Smooth animations
- No generic admin dashboard
- Avoid excessive gradients
- Avoid excessive glassmorphism

7. VIRTUAL PIANO

Implement:

- White and black keys
- Multiple octaves
- Octave selector
- Mouse interaction
- Touch interaction
- Computer keyboard interaction
- Visual key highlighting
- Polyphonic playback
- Sustain support if practical
- Velocity support if practical

Keyboard mapping:

A=C

W=C#

S=D

E=D#

D=E

F=F

T=F#

G=G

Y=G#

H=A

U=A#

J=B

When a key is pressed:

1. Play sound.
2. Create note event.
3. If recording is active, record it.

8. AUDIO ENGINE

Use Tone.js.

Create a centralized AudioEngine:

- Transport
- PianoSampler
- GuitarSampler
- BassSampler
- DrumEngine
- Synth
- Mixer
- Effects

Do NOT scatter Tone.js logic throughout React components.

Expose functions such as:

```
playNote(note, velocity)
releaseNote(note)
scheduleNote(note, time, duration)
setInstrument(instrument)
setBPM(bpm)
startTransport()
pauseTransport()
stopTransport()
```

Audio timing must use Tone.js/Web Audio rather than React render cycles.

9. INSTRUMENT SOUNDS

Initial instruments:

Piano: sampled piano

Bass: sampled bass or suitable synth

Guitar: sampled/suitable synthesized guitar

Drums: individual samples or Tone.js synthesis

Synth: Tone.js synthesizer

Do NOT use copyrighted commercial samples without permission.

Use legally distributable/free assets.
Make sample sources replaceable later.

10. NOTE DATA MODEL

Represent notes as structured musical data.

Example:

```
{  
  id: string,  
  pitch: "C4",  
  midi: 60,  
  start: 0,  
  duration: 0.5,  
  velocity: 0.8  
}
```

Use MIDI note numbers internally where useful.

This enables:

- Editing
- Transposing
- Changing instruments
- Changing tempo
- MIDI export
- Audio rendering

11. RECORDING

Implement:

- Basic recording
- Overdub
- Replace recording
- Quantized recording

When Record is pressed:

1. Start transport timing.
2. Capture played notes.
3. Record pitch, start time, duration, velocity.
4. Stop recording.
5. Convert performance into note objects.
6. Display notes in piano roll.

12. PIANO ROLL

Implement:

- Horizontal time axis
- Vertical pitch axis
- Grid
- Playhead
- Draggable notes
- Create/delete/move/resize/duplicate notes
- Multi-select if practical
- Snap to grid
- Zoom
- Horizontal/vertical scrolling

Interaction:

Click empty space → create note
Drag note → move
Drag edge → resize
Delete → delete
Shift-click → multi-select if implemented

13. QUANTIZATION

Grid options:

1/4
1/8
1/16
1/32

Support free timing and quantized timing.
Quantization must preserve musical information.

14. TRACK SYSTEM

Support multiple tracks.

Each track:

- Name
- Instrument
- Notes/patterns
- Volume
- Mute

- Solo

Tracks must be dynamic; do not hard-code four tracks.

15. PATTERN SYSTEM

Implement reusable patterns and sections:

Intro
Verse
Chorus
Bridge
Outro

Patterns should be duplicatable and rearrangeable.

16. DRUM MACHINE

Create a step sequencer.

Initial sounds:

- Kick
- Snare
- Closed Hi-Hat
- Open Hi-Hat
- Clap
- Tom

Support:

- Pattern length
- BPM
- Swing if practical
- Play/stop
- Step activation/deactivation

17. CHORD SYSTEM

Implement:

- Major
- Minor
- Diminished
- Augmented
- 7th
- Major 7th
- Minor 7th
- Sus2
- Sus4

Examples:

C = C E G
Cm = C Eb G
Am = A C E
F = F A C
G = G B D

Selecting a chord:

- Show notes
- Play chord
- Add to piano roll/timeline

18. SCALE SYSTEM

Implement:

- Major
- Natural Minor
- Harmonic Minor
- Major Pentatonic
- Minor Pentatonic
- Blues

Allow key/scale selection and highlight scale notes on the piano.

19. CHORD SUGGESTIONS

Implement a simple rule-based system.

Do NOT use AI initially.

Example:

C → Am → F
Suggested: G

Display:

"Try G Major"
[Play] [Add]

Keep logic modular for a future recommendation engine.

20. METRONOME

Implement a Tone.js metronome.

Features:

- On/off
- BPM
- Volume
- Time signature

Default:

120 BPM

4/4

21. TRANSPORT

Centralized:

Play

Pause

Stop

Record

Loop

Display:

- BPM
- Current bar
- Current beat
- Current time

Keep piano-roll playhead synchronized with the audio engine.

22. MIXER

Each track:

- Volume
- Mute
- Solo
- Optional pan

Architecture should allow future:

- Reverb
- Delay
- EQ
- Compression

23. MUSIC PLAYER

Implement:

- Play
- Pause
- Seek
- Volume
- Previous
- Next
- Queue
- Playlist

24. PLAY ALONG MODE

User selects:

Song + Instrument

Background song and virtual instrument must be independently controlled.

Do not scrape or illegally download copyrighted music.

Support:

- User-uploaded audio
- Legally licensed audio
- Public-domain audio
- Future legitimate API integrations

25. PROJECT MANAGEMENT

Users can:

- Create
- Save
- Open
- Rename
- Duplicate
- Delete projects

Project contains:

- Metadata
- BPM

- Time signature
- Tracks
- Notes
- Patterns
- Arrangement

26. LOCAL-FIRST STORAGE

The application must work entirely without an account.

Use IndexedDB for full project persistence where practical.

Use localStorage for:

- UI preferences
- Small settings
- Last-opened project ID

Browser refresh must not destroy saved projects.

27. NO AUTHENTICATION / NO CLOUD ACCOUNT

Do NOT implement:

- Login
- Signup
- Password reset
- User accounts
- Social login
- Supabase Auth
- Cloud authentication
- Cloud synchronization
- User profiles

If cloud storage is ever added later, hide it behind a storage abstraction so the core music engine never depends on it.

28. EXPORT

Implement:

1. Project JSON export
2. MIDI export if practical
3. WAV/audio export if practical

Do not create fake export buttons.

Mark unavailable export types as planned.

29. RESPONSIVE DESIGN

Desktop is primary.

Support tablet and mobile.

Mobile:

- Touch piano
- Drawer sidebar
- Horizontal piano-roll scrolling
- Mixer panel
- Accessible controls

30. ACCESSIBILITY

Support:

- Keyboard navigation
- Focus states
- ARIA labels
- Tooltips
- Good contrast
- Clear labels

Users should not need music theory knowledge.

31. ONBOARDING

First launch:

"Welcome to Music Closer ■"

"Play something. Create something. Stay close to music."

Buttons:

[Play an Instrument]
[Create a Song]
[Explore]

No account required.

32. ERROR HANDLING

Handle:

- AudioContext initialization
- Browser autoplay restrictions
- Missing samples

- Audio loading failures
- Invalid project data
- Save failures
- Import failures

Errors must be understandable.
Never silently fail.

33. PERFORMANCE

Important:

- Do not schedule audio through React state updates.
- Do not rerender the entire piano roll for one note change.
- Avoid thousands of unnecessary DOM nodes.
- Use optimized rendering.
- Keep UI state separate from audio state.
- Use Tone.js scheduling for musical timing.

34. CODE STRUCTURE

Suggested:

```
src/
  components/
    piano/
      pianoRoll/
      drums/
      mixer/
      timeline/
      player/
      chords/
      common/
    audio/
      AudioEngine.ts
      instruments/
      transport/
      mixer/
    music/
      notes.ts
      chords.ts
      scales.ts
      quantization.ts
      theory.ts
    store/
      projectStore.ts
      transportStore.ts
      uiStore.ts
    services/
      storage.ts
      audioAssets.ts
    types/
      music.ts
      project.ts
      audio.ts
    utils/
```

Keep components small and reusable.

35. DEVELOPMENT STRATEGY

DO NOT implement the entire application in one pass.

Implement phase-by-phase.

After every phase:

1. Run application.
2. Test acceptance criteria.
3. Fix errors.
4. Check previous phases.
5. Only then proceed.

Never sacrifice existing functionality.

PHASE 1 — CORE PIANO

Implement:

- Project setup
- App shell
- Dark UI
- Piano UI
- Tone.js
- Piano sound
- Keyboard controls

- Octave switching
- Mouse/touch interaction

ACCEPTANCE CRITERIA:

- [] Application starts without build errors.
- [] Application loads in a browser.
- [] Piano resembles a real keyboard.
- [] White keys produce audible notes.
- [] Black keys produce audible notes.
- [] Computer keyboard controls piano.
- [] Multiple notes can be played without crashing.
- [] Octave can be changed.
- [] Pressed keys have visible feedback.
- [] AudioContext initializes correctly after user interaction.
- [] No critical browser-console errors.

PHASE 2 – RECORDING AND TRANSPORT

Implement:

- Note event system
- Recording
- Playback
- Transport
- Metronome
- BPM
- Play/pause/stop
- Loop

ACCEPTANCE CRITERIA:

- [] Record starts correctly.
- [] Notes played during recording are captured.
- [] Each note has pitch, start time, duration, and velocity.
- [] Stop ends recording correctly.
- [] Recorded performance can be played back.
- [] Playback timing is consistent.
- [] Play/pause/stop work.
- [] BPM changes.
- [] Metronome follows BPM.
- [] Loop playback works.
- [] Recording does not depend on React render timing.
- [] Phase 1 functionality still works.

PHASE 3 – PIANO ROLL

Implement:

- Piano roll
- Grid
- Note creation
- Note deletion
- Dragging
- Resizing
- Quantization
- Zoom
- Playhead

ACCEPTANCE CRITERIA:

- [] Recorded notes appear correctly.
- [] Notes reflect timing and pitch.
- [] Empty space creates a note.
- [] Notes can be moved.
- [] Notes can be resized.
- [] Notes can be deleted.
- [] Notes snap to selected grid.
- [] Quantization works.
- [] Playhead syncs with playback.
- [] Editing changes playback.
- [] Normal projects remain responsive.
- [] Previous functionality still works.

PHASE 4 – MULTI-INSTRUMENTS AND DRUM MACHINE

Implement:

- Track system
- Piano
- Bass
- Guitar
- Drums

- Drum step sequencer
- Instrument switching

ACCEPTANCE CRITERIA:

- ☐ Multiple tracks can be created.
- ☐ Each track can select an instrument.
- ☐ Piano sounds correctly.
- ☐ Bass produces bass sound.
- ☐ Guitar produces guitar sound.
- ☐ Drums produce drum sounds.
- ☐ Drum sequencer displays steps.
- ☐ Steps can be activated/deactivated.
- ☐ Drum pattern loops correctly.
- ☐ Tracks can be muted.
- ☐ Tracks can be soloed.
- ☐ Multiple tracks play simultaneously.
- ☐ Existing piano-roll functionality works.

PHASE 5 – PATTERNS AND ARRANGEMENT

Implement:

- Patterns
- Pattern duplication
- Sections
- Intro
- Verse
- Chorus
- Bridge
- Outro
- Arrangement timeline

ACCEPTANCE CRITERIA:

- ☐ Patterns can be created.
- ☐ Patterns can be duplicated.
- ☐ Patterns can be placed multiple times.
- ☐ Sections can be created.
- ☐ Sections can be reordered.
- ☐ Patterns remain synchronized.
- ☐ Playback follows arrangement order.
- ☐ Arrangement looping works.
- ☐ Pattern editing behaves predictably.
- ☐ Existing tracks/instruments still work.

PHASE 6 – CHORDS, SCALES, AND MUSIC THEORY

Implement:

- Chord library
- Major/minor/etc.
- Scale library
- Piano highlighting
- Chord insertion
- Chord suggestions

ACCEPTANCE CRITERIA:

- ☐ Major chords work.
- ☐ Minor chords work.
- ☐ Diminished chords work.
- ☐ Augmented chords work.
- ☐ 7th chords work.
- ☐ Major 7th chords work.
- ☐ Minor 7th chords work.
- ☐ Sus2 and Sus4 work.
- ☐ Selecting a chord displays notes.
- ☐ Chord can be played.
- ☐ Chord can be inserted into piano roll.
- ☐ Major scale works.
- ☐ Natural minor works.
- ☐ Pentatonic scales work.
- ☐ Blues scale works.
- ☐ Scale notes are highlighted.
- ☐ Chord suggestions are musically valid according to implemented rules.
- ☐ Note editing remains functional.

PHASE 7 – MIXER AND PROJECT PERSISTENCE

Implement:

- Mixer

- Volume
- Mute
- Solo
- Optional pan
- Local project saving
- Project loading
- Project management

ACCEPTANCE CRITERIA:

- ☐ Each track has independent volume.
- ☐ Mute works.
- ☐ Solo works.
- ☐ Projects save locally.
- ☐ Browser refresh does not destroy local projects.
- ☐ Projects load correctly.
- ☐ Projects can be renamed.
- ☐ Projects can be duplicated.
- ☐ Projects can be deleted.
- ☐ Musical data survives reload.
- ☐ Invalid project data is handled safely.
- ☐ No authentication is required.
- ☐ No cloud backend is required.
- ☐ Previous functionality still works.

PHASE 8 – MUSIC PLAYER AND PLAY ALONG

Implement:

- Music player
- Playlist
- Queue
- User audio upload
- Play/pause/seek
- Volume
- Play Along Mode

ACCEPTANCE CRITERIA:

- ☐ Supported audio can be loaded.
- ☐ Audio plays.
- ☐ Audio pauses.
- ☐ Audio can be seeked.
- ☐ Volume works.
- ☐ Multiple songs can be queued.
- ☐ Previous/next works.
- ☐ User can play an instrument while background audio plays.
- ☐ Background audio does not stop when instrument is played.
- ☐ Instrument and player volume are independent.
- ☐ Copyright-protected workflows are not implemented illegally.

PHASE 9 – EXPORT AND SHARING

Implement:

- JSON export
- MIDI export
- Audio export where practical
- Local project file sharing/import

ACCEPTANCE CRITERIA:

- ☐ Project exports as JSON.
- ☐ JSON contains required musical information.
- ☐ JSON can be imported back.
- ☐ MIDI export contains correct notes/timing where implemented.
- ☐ Audio export produces a valid playable file where implemented.
- ☐ User can share a project file locally.
- ☐ Imported projects are validated before loading.
- ☐ Export/import failures are clearly reported.
- ☐ No account is required.

FINAL ACCEPTANCE TEST

The complete application passes only when a new user can:

1. Open Music Closer.
2. Click "Play an Instrument."
3. Play a C major chord.
4. Hear a realistic piano sound.
5. Start recording.
6. Play C → E → G → E.

7. Stop recording.
8. See notes in piano roll.
9. Move one note.
10. Change its duration.
11. Press Play.
12. Hear the edited performance.
13. Add a drum track.
14. Create a kick/snare pattern.
15. Add a bass track.
16. Create an A minor chord.
17. Insert it into the timeline.
18. Arrange Intro → Verse → Chorus.
19. Press Play.
20. Hear all tracks synchronized.
21. Change BPM.
22. Confirm synchronization remains correct.
23. Save the project locally.
24. Refresh browser.
25. Reload project.
26. Confirm project is intact.
27. Enter Play Along mode.
28. Play piano while background music plays.
29. Export the project.
30. Import the exported project.
31. Confirm musical structure remains intact.

If any core flow fails, the application is NOT considered complete.

TESTING REQUIREMENTS

After every phase:

- Run development server.
- Check browser console.
- Test acceptance criteria.
- Fix errors.
- Retest previous phases.

Create automated tests where practical for:

- Note calculations
- Chord calculations
- Scale calculations
- Quantization
- Project serialization/deserialization
- Transport calculations
- Local project persistence

Audio interaction should receive manual browser testing.

IMPORTANT IMPLEMENTATION RULES

1. Build real functionality, not mockups.
2. Do not create buttons that do nothing.
3. Do not fake audio functionality.
4. Do not use copyrighted commercial samples without permission.
5. Keep musical data separate from audio rendering.
6. Keep Tone.js logic centralized.
7. Use TypeScript types throughout.
8. Do not over-engineer the MVP.
9. Do not rewrite working functionality unnecessarily.
10. Do not introduce dependencies without a reason.
11. Do not silently ignore errors.
12. If a feature cannot be completed in the current phase, implement clean architecture and document it as future work.
13. Never claim a phase passes unless its acceptance criteria have actually been tested.
14. Preserve all existing functionality when implementing later phases.
15. Do not add authentication, accounts, Supabase, or cloud synchronization.

START NOW

Begin with PHASE 1.

First:

1. Inspect the project directory.
2. Determine whether a project already exists.
3. Preserve useful existing work.
4. Set up the required architecture.
5. Install only necessary dependencies.
6. Implement the application shell.

7. Implement the virtual piano.
8. Integrate Tone.js.
9. Add real playable piano audio.
10. Add computer-keyboard controls.
11. Add octave switching.
12. Run the application.
13. Test every Phase 1 acceptance criterion.
14. Fix all issues found.
15. Only after Phase 1 passes, prepare for Phase 2.

At the end of every phase, provide:

PHASE:
Status: PASS / FAIL

Implemented:
- ...

Acceptance criteria:
- [PASS] ...
- [FAIL] ...

Known issues:
- ...

Next phase:
- ...

FINAL PRINCIPLE:
The goal is not to build another complicated professional DAW.

The goal is to build something that makes a person want to sit down, press a key, hear music, and think:

"I can make something."